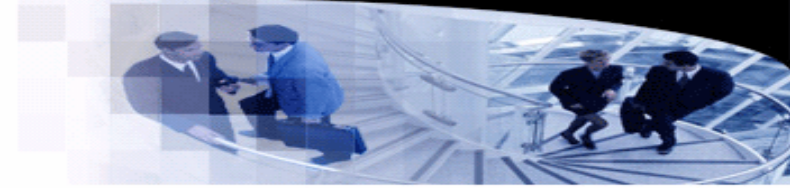




武汉大学
WUHAN UNIVERSITY



Programming Languages and Compilers

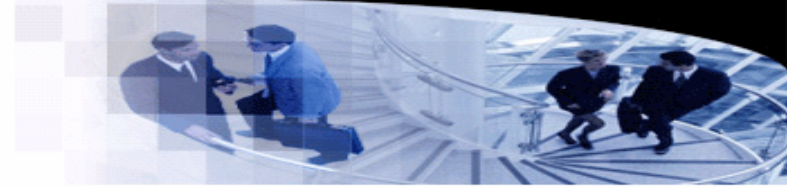
Lecture 14. LL(k) Parsing

袁梦霆 (YUAN Mengting)

ymt@whu.edu.cn

School of Computer Science, Wuhan University

Top-down Parsing



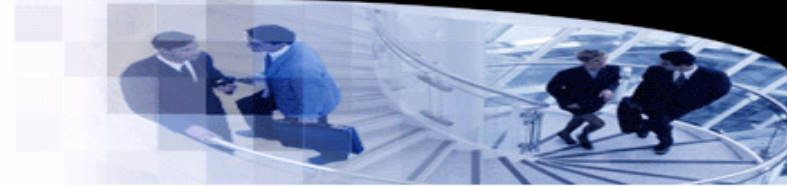
- Top-down parsing via pushdown automata

- Push “ $\$S$ ” into the stack.
- If the top of the stack is a nonterminal A , replace A with the right-hand-side of the production nondeterministically.
- If the top of the stack is a terminal a and
 - the input symbol is a as well, pop a from the stack.
 - the input symbol is not a , the copy rejects the input.
- If the input symbol is $\$$ (which implies the end of the input) and the top of the stack is $\$$, the input is accepted.

- Issues

- Suppose there is a production $A \rightarrow A\alpha$. When A is placed at the top of the stack, A will be replaced with $A\alpha$. A is again at the top of the stack, which leads to an infinite loop.
- How to avoid the performance issues caused by nondeterministic choices?

Top-down Parsing



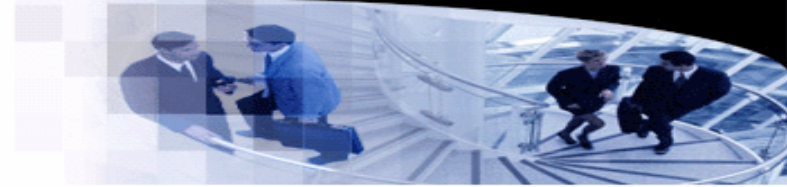
- Elimination of left recursion

- **Definition:** A grammar is **left-recursive**, if there exists a nonterminal A such that $A \Rightarrow^+ A\alpha$.

- Eliminating direct left recursion

- Consider a set of productions $P \rightarrow P\alpha|\beta$, where β does not start with P .
- Observation: $P \Rightarrow^+ \beta\alpha^*$.
- Therefore, the left recursion can be eliminated by a set of equivalent productions: $P \rightarrow \beta P', P' \rightarrow \alpha P'|\epsilon$.
- A more general version: $P \rightarrow P\alpha_1|P\alpha_2|\dots|P\alpha_m|\beta_1|\beta_2|\dots|\beta_n$.
- Solution: $P \rightarrow \beta_1 P'|\beta_2 P'|\dots|\beta_n P', P' \rightarrow \alpha_1 P'|\alpha_2 P'|\dots|\alpha_m P'|\epsilon$.

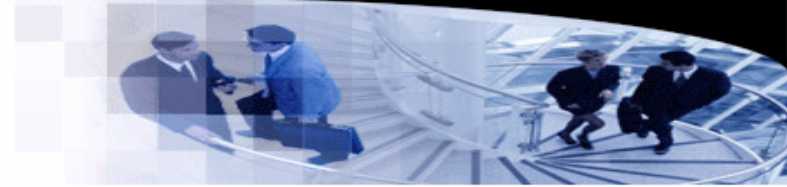
Top-down Parsing



- Eliminating indirect left recursion

- Consider a grammar with **indirect left recursion**.
 - $S \rightarrow Qc|c$
 - $Q \rightarrow Rb|b$
 - $R \rightarrow Sa|a$
- **Observation:** The start symbols of the derivation $S \Rightarrow Q \dots \Rightarrow R \dots \Rightarrow S \dots$ forms a cycle.
- The indirect recursion can be eliminated by breaking these cycles.
 - Arrange all nonterminals in the grammar as $P_1, P_2, \dots, P_n$.
 - For each production $P_i \rightarrow P_j\gamma$, where $i > j$, replace P_j with P_j 's right-hand side strings.
 - The replacement would cause direct left recursions, which can be eliminated by using the previous method.

Top-down Parsing



- Eliminating indirect left recursion

- Example:

- $P_1: S \rightarrow Qc|c$
 - $P_2: Q \rightarrow Rb|b$
 - $P_3: R \rightarrow Sa|a$

- The production $R \rightarrow Sa$ (i.e., $P_3 \rightarrow P_1 a$) may cause indirect left recursion.
 - We replace S by $Qc|c$ to obtain $R \rightarrow Qca|ca|a$.
 - Since $R \rightarrow Qca$ may also cause left recursion, we replace Q by $Rb|b$ to obtain $R \rightarrow Rbca|bca|ca|a$.
 - We eliminate the direct left recursion to obtain:
 - $R \rightarrow bcaR'|caR'|aR'$
 - $R' \rightarrow bcaR'|\epsilon$

Top-down Parsing

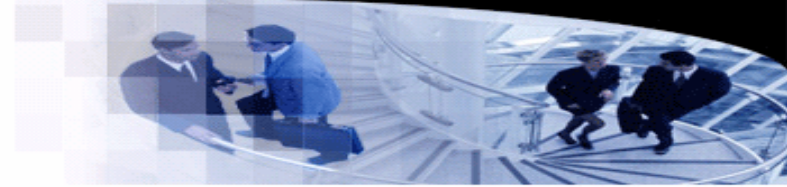
- Eliminating indirect left recursion

- The grammar of expressions:

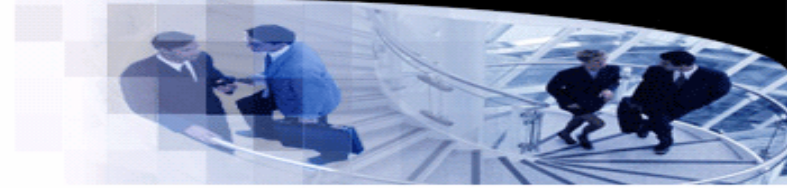
- $E \rightarrow E + T \mid T$
- $T \rightarrow T * F \mid F$
- $F \rightarrow (E) \mid i$

- After elimination:

- $E \rightarrow TE'$
- $E' \rightarrow +TE' \mid \epsilon$
- $T \rightarrow FT'$
- $T' \rightarrow * FT' \mid \epsilon$
- $F \rightarrow (E) \mid i$



LL(k) Parsing



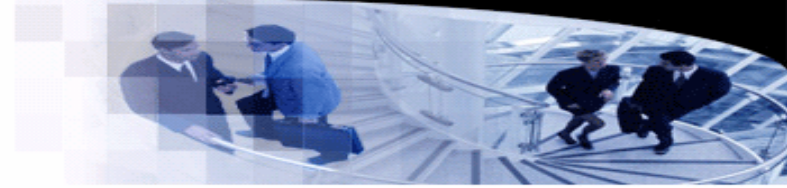
- Eliminating nondeterministic choices

- The top-down parsing algorithm suffers from the performance issue caused by nondeterministic choices.
- Consider $A \rightarrow \gamma_1 | \gamma_2 | \dots | \gamma_n$. When A is at the top of the stack, which production should we choose to derive?

- LL(k) parsing

- Idea: We look at the next k symbols in the input to predict the “correct” production.
- The algorithm is call LL(k) parsing algorithm when $k = 1$.
- LL(1) and LL(2) are most commonly used.

LL(1) Parsing



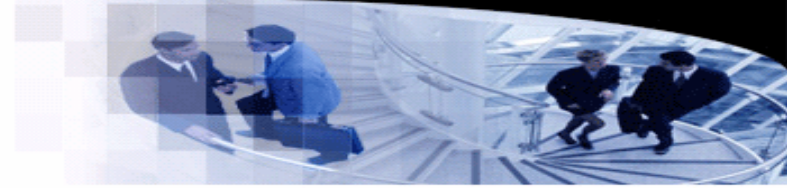
- Sketch of LL(1)

- 1. Push S to the stack.
- 2. Read the input from the left to the right, and match the following cases:
 - If the top of the stack is a terminal x , match x with the first input symbol a . If $x = a$, then pop x and read the next token. Report an error if $x \neq a$.
 - If the top of the stack is a nonterminal A and the input symbol is a , then select a production $A \rightarrow \gamma_i$ such that $\gamma_i \Rightarrow^* a\beta$. Report an error if there is no such γ_i . (What about $A \rightarrow \gamma_i \Rightarrow^* \epsilon$?)
 - If the stack is empty, then check the input. Return ACCEPT if the input is empty. Report an error otherwise.
- 3. Repeat 2.

- Key problem

- How to select a production $A \rightarrow \gamma_i$?

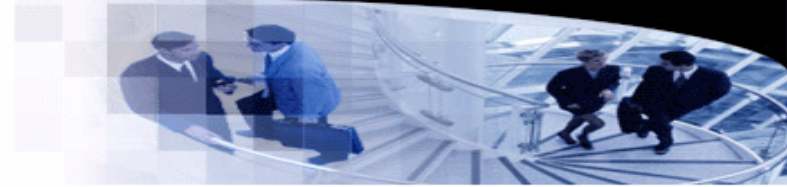
LL(1) Parsing



- First and Follow function

- We introduce the *First* function to help us predict the first symbol a string may derive.
- $First(\alpha) = \{a \mid \alpha \Rightarrow^* a\beta, a \in V_T, \beta \in (V_N \cup V_T)^*\}$.
- $\epsilon \in First(a)$ iff $\alpha \Rightarrow^* \epsilon$.
- Given $A \rightarrow \gamma_1 | \gamma_2 | \dots | \gamma_n$, when A is on the top of the stack and a is the input symbol, the production $A \rightarrow \gamma_i$ is selected if $a \in First(\gamma_i)$.
- If no such γ_i is selected, report an error, because none of these productions can derive a string starting with a .
- If more than one production are selected, then the grammar is not LL(1), as the nondeterminism cannot be overcome.

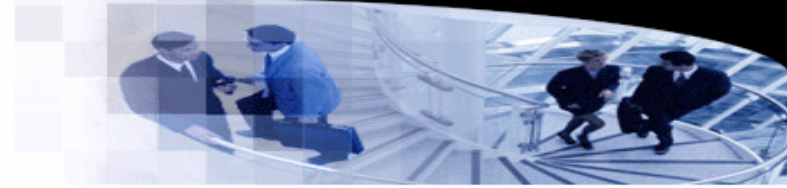
LL(1) Parsing



- First and Follow function

- Let us consider a production $A \rightarrow \gamma_i \Rightarrow^* \epsilon$.
- If the production is selected, A will eventually be replaced by the empty string ϵ .
- It seems that A cannot derive a string starting with a .
- When should this production be selected? (Every production counts when we define a language!)
- Now let us check the stack. Suppose the configuration of the stack is $AX\beta$, where A is on the top.
- If $A \rightarrow \gamma_i \Rightarrow^* \epsilon$ is selected, we know A will eventually be replaced with ϵ . It is X that will be used to match a .
- As X can be a terminal or nonterminal, we introduce the *Follow* function to describe all possible symbols that may occur **following** A in a derived string.
- $Follow(A) = \{a \mid S \Rightarrow^* \alpha A a \beta, a \in V_T, \alpha, \beta \in (V_N \cup V_T)^*\}$.
- $\$ \in Follow(A)$ if $S \Rightarrow^* \alpha A$.

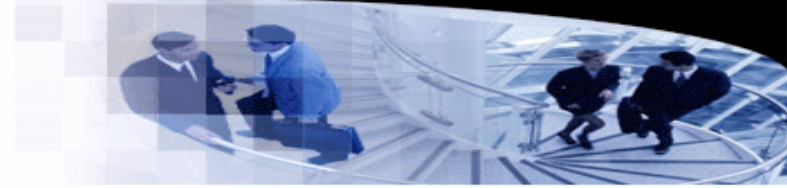
LL(1) Parsing



- Selecting a production

- Consider $A \rightarrow \gamma_1 | \gamma_2 | \dots | \gamma_n$, when A is on the top of the stack and a is the input symbol.
- The production $A \rightarrow \gamma_i$ is selected if $a \in First(\gamma_i)$.
- The production $A \rightarrow \gamma_j$ is selected if $\epsilon \in First(\gamma_j)$ and $a \in Follow(A)$.
- If no such γ_i is selected, report an error, because none of these productions can derive a string starting with a .
- If more than one production are selected, then the grammar is **NOT** LL(1), as the nondeterminism cannot be overcome.

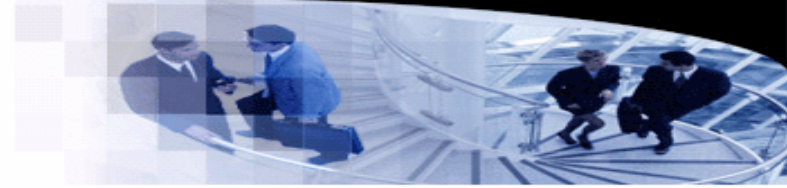
LL(1) Parsing



- Calculating the *First* function

- Given a symbol A , $First(A)$ is calculated by applying the following rules, until the result set does not change.
 - If $A \in V_T$, then $First(A) = \{A\}$.
 - If $A \rightarrow \epsilon$, then $\epsilon \in First(A)$.
 - If A is a nonterminal and $A \rightarrow X_1X_2 \dots X_n$, then
 - $First(X_1) \setminus \{\epsilon\} \subseteq First(A)$,
 - $First(X_i) \setminus \{\epsilon\} \subseteq First(A)$, if there exists an i such that $\forall j < i (\epsilon \notin First(X_j))$,
 - and $\epsilon \in First(A)$ if $\forall j \leq n (\epsilon \in First(X_j))$.
- Given a string $\alpha = X_1X_2 \dots X_n$, $First(\alpha)$ is calculated by applying the following rules, until the result set does not change.
 - $First(X_1) \setminus \{\epsilon\} \subseteq First(\alpha)$,
 - $First(X_i) \setminus \{\epsilon\} \subseteq First(\alpha)$, if there exists an i such that $\forall j < i (\epsilon \notin First(X_j))$,
 - and $\epsilon \in First(\alpha)$ if $\forall j \leq n (\epsilon \in First(X_j))$.

LL(1) Parsing



- Calculating the *Follow* function

- Given a nonterminal A , $Follow(A)$ is calculated by applying the following rules, until the result set does not change.
 - If A is the start symbol S , then $\$ \in Follow(A)$.
 - If $B \rightarrow \alpha A \beta$, then $First(\beta) \setminus \{\epsilon\} \subseteq Follow(A)$.
 - If $B \rightarrow \alpha A \beta$ and $\epsilon \in First(\beta)$, then $Follow(B) \in Follow(A)$.

- Algorithm

- **let** $First = \lambda x. \emptyset$, $Follow = \lambda x. \emptyset$
- **do** {
 - **foreach** $p \in P$ **do** // Enumerate all productions.
 - update the *First* and *Follow* function by the rules.
 - **endfor**
- **}** **while** *First* or *Follow* has been changed.

LL(1) Parsing

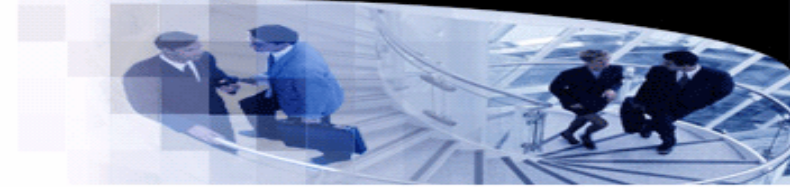
- Example of *First* and *Follow*

- Grammar $G[E]$:

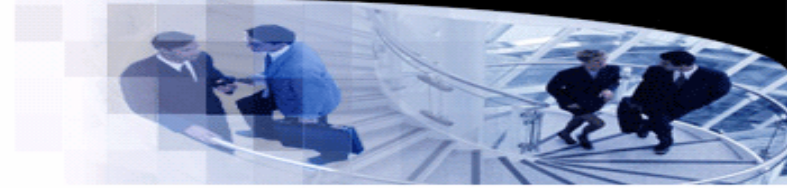
- $E \rightarrow TE'$
- $E' \rightarrow +TE' \mid \epsilon$
- $T \rightarrow FT'$
- $T' \rightarrow * FT' \mid \epsilon$
- $F \rightarrow (E) \mid i$

- *First* functions:

- $First(E) = \{ \quad \}$
- $First(E') = \{ \quad \}$
- $First(T) = \{ \quad \}$
- $First(T') = \{ \quad \}$
- $First(F) = \{ \quad \}$



LL(1) Parsing



• Example of *First* and *Follow*

• Grammar $G[E]$:

- $E \rightarrow TE'$
- $E' \rightarrow +TE' \mid \epsilon$
- $T \rightarrow FT'$
- $T' \rightarrow *FT' \mid \epsilon$
- $F \rightarrow (E) \mid i$

• *First* functions:

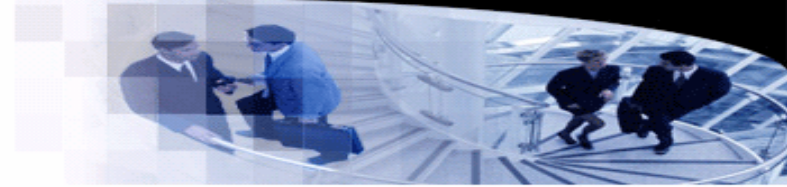
- $First(E) = \{ \quad \}$
- $First(E') = \{+, \epsilon\}$
- $First(T) = \{ \quad \}$
- $First(T') = \{*, \epsilon\}$
- $First(F) = \{ (, i \}$

Introduced by $E' \rightarrow +TE' \mid \epsilon$.

$T' \rightarrow *FT' \mid \epsilon$.

$F' \rightarrow (E) \mid i$.

LL(1) Parsing



- Example of *First* and *Follow*

- Grammar $G[E]$:

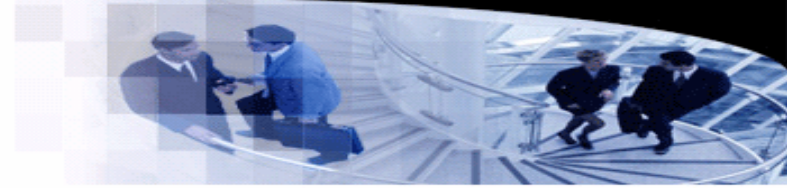
- $E \rightarrow TE'$
- $E' \rightarrow +TE' \mid \epsilon$
- $T \rightarrow FT'$
- $T' \rightarrow *FT' \mid \epsilon$
- $F \rightarrow (E) \mid i$

- *First* functions:

- $First(E) = \{ \}$
- $First(E') = \{+, \epsilon\}$
- $First(T) = \{ (, i \}$
- $First(T') = \{*, \epsilon\}$
- $First(F) = \{ (, i \}$

$T \rightarrow FT'$ implies that $First(F) \subseteq First(T)$.

LL(1) Parsing



- Example of *First* and *Follow*

- Grammar $G[E]$:

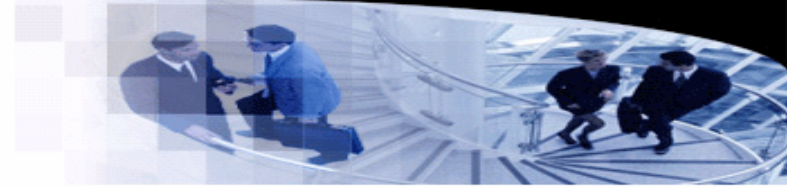
- $E \rightarrow TE'$
- $E' \rightarrow +TE' \mid \epsilon$
- $T \rightarrow FT'$
- $T' \rightarrow *FT' \mid \epsilon$
- $F \rightarrow (E) \mid i$

- *First* functions:

- $First(E) = \{(, i\}$
- $First(E') = \{+, \epsilon\}$
- $First(T) = \{(, i\}$
- $First(T') = \{*, \epsilon\}$
- $First(F) = \{(, i\}$

$E \rightarrow TE'$ implies that $First(T) \subseteq First(E)$.

LL(1) Parsing



- Example of *First* and *Follow*

- Grammar $G[E]$:

- $E \rightarrow TE'$
- $E' \rightarrow +TE' \mid \epsilon$
- $T \rightarrow FT'$
- $T' \rightarrow * FT' \mid \epsilon$
- $F \rightarrow (E) \mid i$

- *First* functions:

- $First(E) = \{ (, i \}$
- $First(E') = \{ +, \epsilon \}$
- $First(T) = \{ (, i \}$
- $First(T') = \{ *, \epsilon \}$
- $First(F) = \{ (, i \}$

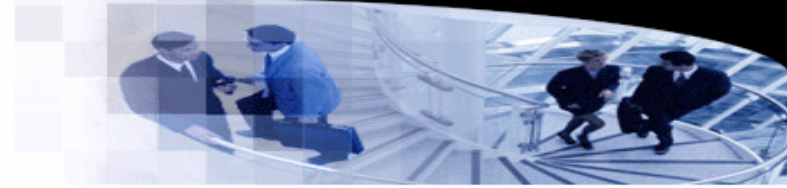
- *First* functions (for the right-hand side of productions)

- $First(TE') = First(T) = \{ (, i \}$
- $First(+TE') = \{ + \}, First(\epsilon) = \{ \epsilon \}$
- $First(FT') = First(F) = \{ (, i \}$
- $First(* FT') = \{ * \}, First(\epsilon) = \{ \epsilon \}$
- $First((E)) = \{ (\}, First(i) = \{ i \}$

- *Follow* functions

- $Follow(E) = \{ \quad \}$
- $Follow(E') = \{ \quad \}$
- $Follow(T) = \{ \quad \}$
- $Follow(T') = \{ \quad \}$
- $Follow(F) = \{ \quad \}$

LL(1) Parsing



- Example of *First* and *Follow*

- Grammar $G[E]$:

- $E \rightarrow TE'$
- $E' \rightarrow +TE' \mid \epsilon$
- $T \rightarrow FT'$
- $T' \rightarrow *FT' \mid \epsilon$
- $F \rightarrow (E) \mid i$

- *First* functions:

- $First(E) = \{ (, i \}$
- $First(E') = \{ +, \epsilon \}$
- $First(T) = \{ (, i \}$
- $First(T') = \{ *, \epsilon \}$
- $First(F) = \{ (, i \}$

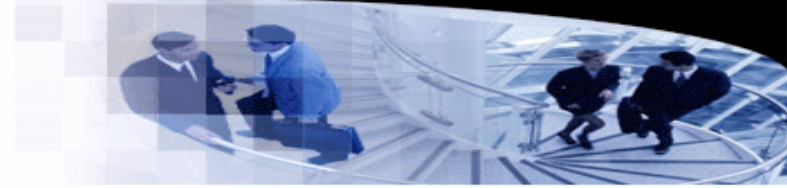
- *First* functions (for the right-hand side of productions)

- $First(TE') = First(T) = \{ (, i \}$
- $First(+TE') = \{ + \}, First(\epsilon) = \{ \epsilon \}$
- $First(FT') = First(F) = \{ (, i \}$
- $First(*FT') = \{ * \}, First(\epsilon) = \{ \epsilon \}$
- $First((E)) = \{ (\}, First(i) = \{ i \}$

- *Follow* functions

- $Follow(E) = \{ \$ \}$ — ϵ is the start symbol.
- $Follow(E') = \{ \quad \}$
- $Follow(T) = \{ \quad \}$
- $Follow(T') = \{ \quad \}$
- $Follow(F) = \{ \quad \}$

LL(1) Parsing



• Example of *First* and *Follow*

• Grammar $G[E]$:

- $E \rightarrow TE'$
- $E' \rightarrow +TE' \mid \epsilon$
- $T \rightarrow FT'$
- $T' \rightarrow * FT' \mid \epsilon$
- $F \rightarrow (E) \mid i$

• *First* functions:

- $First(E) = \{ (, i \}$
- $First(E') = \{ +, \epsilon \}$
- $First(T) = \{ (, i \}$
- $First(T') = \{ *, \epsilon \}$
- $First(F) = \{ (, i \}$

• *First* functions (for the right-hand side of productions)

- $First(TE') = First(T) = \{ (, i \}$
- $First(+TE') = \{ + \}, First(\epsilon) = \{ \epsilon \}$
- $First(FT') = First(F) = \{ (, i \}$
- $First(* FT') = \{ * \}, First(\epsilon) = \{ \epsilon \}$
- $First((E)) = \{ (\}, First(i) = \{ i \}$

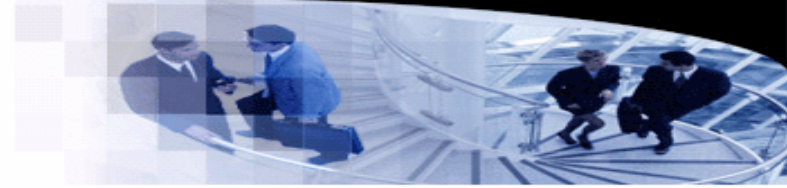
• *Follow* functions

- $Follow(E) = \{ \$ \}$
- $Follow(E') = \{ \$ \}$
- $Follow(T) = \{ +, \$ \}$
- $Follow(T') = \{ \}$
- $Follow(F) = \{ \}$

$Follow(E) \subseteq Follow(E')$.

$First(E') \setminus \{ \epsilon \} \subseteq Follow(T)$.
 $Follow(E) \subseteq Follow(T)$ as
 $\epsilon \in First(E')$.

LL(1) Parsing



- Example of *First* and *Follow*

- Grammar $G[E]$:

- $E \rightarrow TF'$

- $E' \rightarrow +TE' \mid \epsilon$ — No updates.

- $T \rightarrow FT'$

- $T' \rightarrow * FT' \mid \epsilon$

- $F \rightarrow (E) \mid i$

- *First* functions:

- $First(E) = \{ (, i \}$

- $First(E') = \{ +, \epsilon \}$

- $First(T) = \{ (, i \}$

- $First(T') = \{ *, \epsilon \}$

- $First(F) = \{ (, i \}$

- *First* functions (for the right-hand side of productions)

- $First(TE') = First(T) = \{ (, i \}$

- $First(+TE') = \{ + \}, First(\epsilon) = \{ \epsilon \}$

- $First(FT') = First(F) = \{ (, i \}$

- $First(* FT') = \{ * \}, First(\epsilon) = \{ \epsilon \}$

- $First((E)) = \{ (\}, First(i) = \{ i \}$

- *Follow* functions

- $Follow(E) = \{ \$ \}$

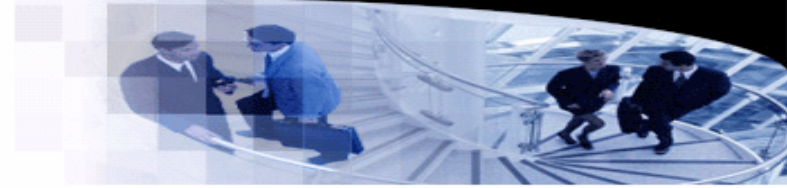
- $Follow(E') = \{ \$ \}$

- $Follow(T) = \{ +, \$ \}$

- $Follow(T') = \{ \quad \}$

- $Follow(F) = \{ \quad \}$

LL(1) Parsing



• Example of *First* and *Follow*

• Grammar $G[E]$:

- $E \rightarrow TE'$
- $E' \rightarrow +TE' \mid \epsilon$
- $T \rightarrow FT'$
- $T' \rightarrow *FT' \mid \epsilon$
- $F \rightarrow (E) \mid i$

• *First* functions:

- $First(E) = \{ (, i \}$
- $First(E') = \{ +, \epsilon \}$
- $First(T) = \{ (, i \}$
- $First(T') = \{ *, \epsilon \}$
- $First(F) = \{ (, i \}$

• *First* functions (for the right-hand side of productions)

- $First(TE') = First(T) = \{ (, i \}$
- $First(+TE') = \{ + \}, First(\epsilon) = \{ \epsilon \}$
- $First(FT') = First(F) = \{ (, i \}$
- $First(*FT') = \{ * \}, First(\epsilon) = \{ \epsilon \}$
- $First((E)) = \{ (\}, First(i) = \{ i \}$

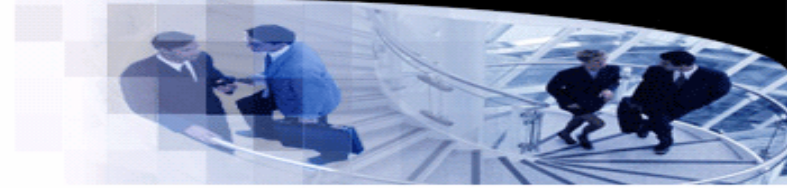
• *Follow* functions

- $Follow(E) = \{ \$ \}$
- $Follow(E') = \{ \$ \}$
- $Follow(T) = \{ +, \$ \}$
- $Follow(T') = \{ +, \$ \}$
- $Follow(F) = \{ +, *, \$ \}$

$Follow(T) \subseteq Follow(T').$

$First(T') \setminus \{ \epsilon \} \subseteq Follow(F).$
 $Follow(T) \subseteq Follow(F)$ as $\epsilon \in First(E').$

LL(1) Parsing



- Example of *First* and *Follow*

- Grammar $G[E]$:

- $E \rightarrow TE'$
- $E' \rightarrow +TE' \mid \epsilon$
- $T \rightarrow FT'$
- $T' \rightarrow *FT' \mid \epsilon$
- $F \rightarrow (E) \mid i$

No updates.

- *First* functions:

- $First(E) = \{ (, i \}$
- $First(E') = \{ +, \epsilon \}$
- $First(T) = \{ (, i \}$
- $First(T') = \{ *, \epsilon \}$
- $First(F) = \{ (, i \}$

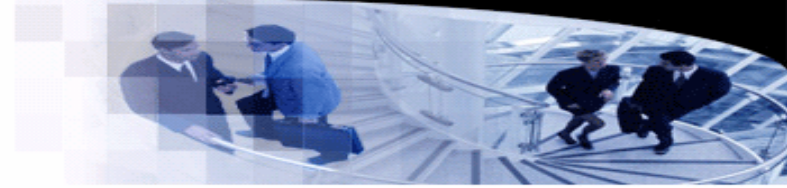
- *First* functions (for the right-hand side of productions)

- $First(TE') = First(T) = \{ (, i \}$
- $First(+TE') = \{ + \}, First(\epsilon) = \{ \epsilon \}$
- $First(FT') = First(F) = \{ (, i \}$
- $First(*FT') = \{ * \}, First(\epsilon) = \{ \epsilon \}$
- $First((E)) = \{ (\}, First(i) = \{ i \}$

- *Follow* functions

- $Follow(E) = \{ \$ \}$
- $Follow(E') = \{ \$ \}$
- $Follow(T) = \{ +, \$ \}$
- $Follow(T') = \{ +, \$ \}$
- $Follow(F) = \{ +, *, \$ \}$

LL(1) Parsing



- Example of *First* and *Follow*

- Grammar $G[E]$:

- $E \rightarrow TE'$
- $E' \rightarrow +TE' \mid \epsilon$
- $T \rightarrow FT'$
- $T' \rightarrow * FT' \mid \epsilon$
- $F \rightarrow (E) \mid i$

- *First* functions:

- $First(E) = \{ (, i \}$
- $First(E') = \{ +, \epsilon \}$
- $First(T) = \{ (, i \}$
- $First(T') = \{ *, \epsilon \}$
- $First(F) = \{ (, i \}$

- *First* functions (for the right-hand side of productions)

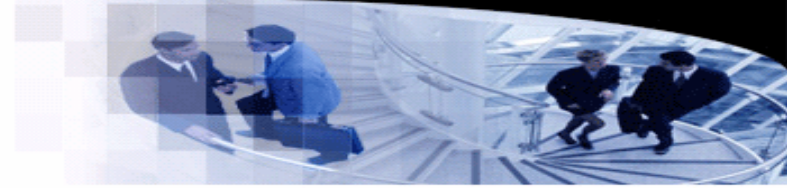
- $First(TE') = First(T) = \{ (, i \}$
- $First(+TE') = \{ + \}, First(\epsilon) = \{ \epsilon \}$
- $First(FT') = First(F) = \{ (, i \}$
- $First(* FT') = \{ * \}, First(\epsilon) = \{ \epsilon \}$
- $First((E)) = \{ (\}, First(i) = \{ i \}$

- *Follow* functions

- $Follow(E) = \{ \$,) \}$
- $Follow(E') = \{ \$ \}$
- $Follow(T) = \{ +, \$ \}$
- $Follow(T') = \{ +, \$ \}$
- $Follow(F) = \{ +, *, \$ \}$

$) \in Follow(E)$

LL(1) Parsing



• Example of *First* and *Follow*

• Grammar $G[E]$:

- $E \rightarrow TE'$
- $E' \rightarrow +TE' \mid \epsilon$
- $T \rightarrow FT'$
- $T' \rightarrow *FT' \mid \epsilon$
- $F \rightarrow (E) \mid i$

• *First* functions:

- $First(E) = \{ (, i \}$
- $First(E') = \{ +, \epsilon \}$
- $First(T) = \{ (, i \}$
- $First(T') = \{ *, \epsilon \}$
- $First(F) = \{ (, i \}$

• *First* functions (for the right-hand side of productions)

- $First(TE') = First(T) = \{ (, i \}$
- $First(+TE') = \{ + \}, First(\epsilon) = \{ \epsilon \}$
- $First(FT') = First(F) = \{ (, i \}$
- $First(*FT') = \{ * \}, First(\epsilon) = \{ \epsilon \}$
- $First((E)) = \{ (\}, First(i) = \{ i \}$

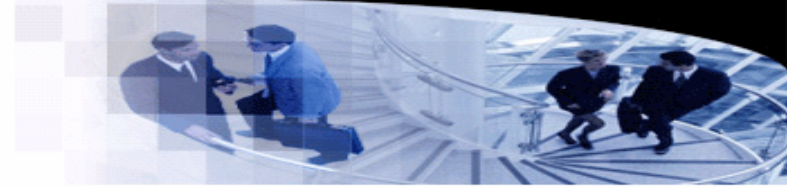
• *Follow* functions

- $Follow(E) = \{ \$,) \}$
- $Follow(E') = \{ \$,) \}$
- $Follow(T) = \{ +, \$,) \}$
- $Follow(T') = \{ +, \$ \}$
- $Follow(F) = \{ +, *, \$ \}$

$Follow(E) \subseteq Follow(E')$

$Follow(E) \subseteq Follow(T)$ as $\epsilon \in First(E')$.

LL(1) Parsing



- Example of *First* and *Follow*

- Grammar $G[E]$:

- $E \rightarrow TE'$

- $E' \rightarrow +TE' \mid \epsilon$ — No updates

- $T \rightarrow FT'$

- $T' \rightarrow * FT' \mid \epsilon$

- $F \rightarrow (E) \mid i$

- *First* functions:

- $First(E) = \{ (, i \}$

- $First(E') = \{ +, \epsilon \}$

- $First(T) = \{ (, i \}$

- $First(T') = \{ *, \epsilon \}$

- $First(F) = \{ (, i \}$

- *First* functions (for the right-hand side of productions)

- $First(TE') = First(T) = \{ (, i \}$

- $First(+TE') = \{ + \}, First(\epsilon) = \{ \epsilon \}$

- $First(FT') = First(F) = \{ (, i \}$

- $First(* FT') = \{ * \}, First(\epsilon) = \{ \epsilon \}$

- $First((E)) = \{ (\}, First(i) = \{ i \}$

- *Follow* functions

- $Follow(E) = \{ \$,) \}$

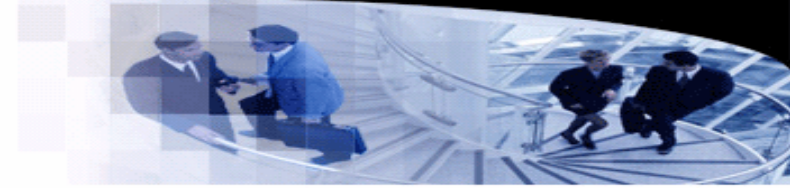
- $Follow(E') = \{ \$,) \}$

- $Follow(T) = \{ +, \$,) \}$

- $Follow(T') = \{ +, \$ \}$

- $Follow(F) = \{ +, *, \$ \}$

LL(1) Parsing



• Example of *First* and *Follow*

• Grammar $G[E]$:

- $E \rightarrow TE'$
- $E' \rightarrow +TE' \mid \epsilon$
- $T \rightarrow FT'$
- $T' \rightarrow *FT' \mid \epsilon$
- $F \rightarrow (E) \mid i$

• *First* functions:

- $First(E) = \{ (, i \}$
- $First(E') = \{ +, \epsilon \}$
- $First(T) = \{ (, i \}$
- $First(T') = \{ *, \epsilon \}$
- $First(F) = \{ (, i \}$

• *First* functions (for the right-hand side of productions)

- $First(TE') = First(T) = \{ (, i \}$
- $First(+TE') = \{ + \}, First(\epsilon) = \{ \epsilon \}$
- $First(FT') = First(F) = \{ (, i \}$
- $First(*FT') = \{ * \}, First(\epsilon) = \{ \epsilon \}$
- $First((E)) = \{ (\}, First(i) = \{ i \}$

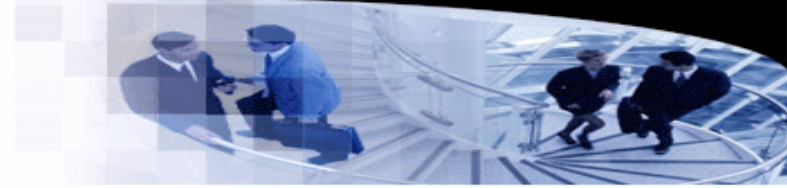
• *Follow* functions

- $Follow(E) = \{ \$,) \}$
- $Follow(E') = \{ \$,) \}$
- $Follow(T) = \{ +, \$,) \}$
- $Follow(T') = \{ +, \$,) \}$
- $Follow(F) = \{ +, *, \$,) \}$

$Follow(T) \subseteq Follow(T')$

$Follow(F) \subseteq Follow(F)$ as $\epsilon \in First(T')$.

LL(1) Parsing



• Example of *First* and *Follow*

• Grammar $G[E]$:

- $E \rightarrow TE'$
- $E' \rightarrow +TE' \mid \epsilon$
- $T \rightarrow FT'$
- $T' \rightarrow *FT' \mid \epsilon$
- $F \rightarrow (E) \mid i$

• *First* functions:

- $First(E) = \{ (, i \}$
- $First(E') = \{ +, \epsilon \}$
- $First(T) = \{ (, i \}$
- $First(T') = \{ *, \epsilon \}$
- $First(F) = \{ (, i \}$

• *First* functions (for the right-hand side of productions)

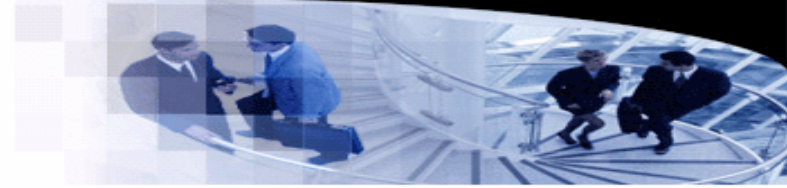
- $First(TE') = First(T) = \{ (, i \}$
- $First(+TE') = \{ + \}, First(\epsilon) = \{ \epsilon \}$
- $First(FT') = First(F) = \{ (, i \}$
- $First(*FT') = \{ * \}, First(\epsilon) = \{ \epsilon \}$
- $First((E)) = \{ (\}, First(i) = \{ i \}$

• *Follow* functions

- $Follow(E) = \{ \$,) \}$
- $Follow(E') = \{ \$,) \}$
- $Follow(T) = \{ +, \$,) \}$
- $Follow(T') = \{ +, \$,) \}$
- $Follow(F) = \{ +, *, \$,) \}$

No updates in the last iteration.

LL(1) Parsing



- Table-driven parsing

- When A is on the top of the stack, a production of A is selected according to the input symbol a with the help of the *First* and *Follow* function.
- This selection process can be formalized as a function $Select: V_N \times V_T \rightarrow P$, which selects a production based on the top stack symbol and the input symbol.
- However, calculating *First* and *Follow* for each parsing step is time-consuming.
- It is more efficient to calculate the *Select* function before parsing, as the function is fixed for a given grammar.
- Therefore, we can calculate the *Select* function and store the function in a table.
- $Select(A, a) = "A \rightarrow \gamma"$, if
 - $a \in First(\gamma)$
 - or $\epsilon \in First(\gamma)$ while $a \in Follow(A)$.

LL(1) Parsing

- Table-driven parsing

- Example of LL(1) parsing table:

	i	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'						
T						
T'						
F						

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \epsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \epsilon$

$F \rightarrow (E) \mid i$

• $First(TE') = First(T) = \{(, i\}$

• $First(+TE') = \{+\}, First(\epsilon) = \{\epsilon\}$

• $First(FT') = First(F) = \{(, i\}$

• $First(*FT') = \{*\}, First(\epsilon) = \{\epsilon\}$

• $First((E)) = \{(, First(i) = \{i\}$

• $Follow(E) = \{\$,)\}$

• $Follow(E') = \{\$,)\}$

• $Follow(T) = \{+, \$,)\}$

• $Follow(T') = \{+, \$,)\}$

• $Follow(F) = \{+, *, \$,)\}$

LL(1) Parsing

- Table-driven parsing

- Example of LL(1) parsing table:

	i	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$				
T						
T'						
F						

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \epsilon$

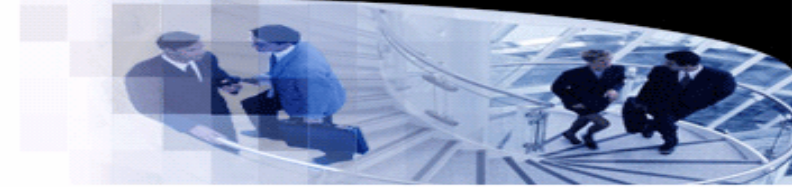
$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \epsilon$

$F \rightarrow (E) \mid i$

- $First(TE') = First(T) = \{ (, i \}$
- $First(+TE') = \{ + \}$ $First(\epsilon) = \{ \epsilon \}$
- $First(FT') = First(F) = \{ (, i \}$
- $First(*FT') = \{ * \}, First(\epsilon) = \{ \epsilon \}$
- $First((E)) = \{ (\}, First(i) = \{ i \}$

- $Follow(E) = \{ \$,) \}$
- $Follow(E') = \{ \$,) \}$
- $Follow(T) = \{ +, \$,) \}$
- $Follow(T') = \{ +, \$,) \}$
- $Follow(F) = \{ +, *, \$,) \}$



LL(1) Parsing

- Table-driven parsing

- Example of LL(1) parsing table:

	i	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T						
T'						
F						

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \epsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \epsilon$

$F \rightarrow (E) \mid i$

- $First(TE') = First(T) = \{ (, i \}$

- $First(+TE') = \{ + \}, First(\epsilon) = \{ \epsilon \}$

- $First(FT') = First(F) = \{ (, i \}$

- $First(*FT') = \{ * \}, First(\epsilon) = \{ \epsilon \}$

- $First((E)) = \{ (\}, First(i) = \{ i \}$

- $Follow(E) = \{ \$,) \}$

- $Follow(E') = \{ \$,) \}$

- $Follow(T) = \{ +, \$,) \}$

- $Follow(T') = \{ +, \$,) \}$

- $Follow(F) = \{ +, *, \$,) \}$

LL(1) Parsing

- Table-driven parsing

- Example of LL(1) parsing table:

	i	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow * FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow i$			$F \rightarrow (E)$		

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \epsilon$

$T \rightarrow FT'$

$T' \rightarrow * FT' \mid \epsilon$

$F \rightarrow (E) \mid i$

- $First(TE') = First(T) = \{ (, i \}$

- $First(+TE') = \{ + \}, First(\epsilon) = \{ \epsilon \}$

- $First(FT') = First(F) = \{ (, i \}$

- $First(* FT') = \{ * \}, First(\epsilon) = \{ \epsilon \}$

- $First((E)) = \{ (\}, First(i) = \{ i \}$

- $Follow(E) = \{ \$,) \}$

- $Follow(E') = \{ \$,) \}$

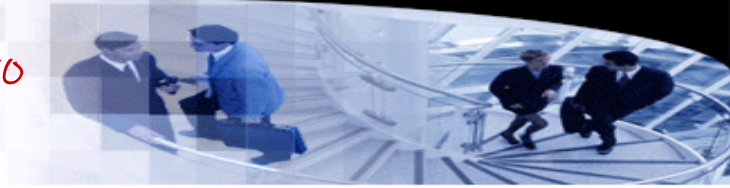
- $Follow(T) = \{ +, \$,) \}$

- $Follow(T') = \{ +, \$,) \}$

- $Follow(F) = \{ +, *, \$,) \}$

LL(1) Parsing

Push \$ and the start symbol E to the stack at the beginning.



- Table-driven parsing
 - Example of LL(1) parsing:

	i	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow i$			$F \rightarrow (E)$		

[illegible]

- ~~Example of LL(1) parsing.~~

$$\text{Select}(\mathcal{E}, i) = E \rightarrow TE'.$$

Stack	Input	Action
E\$	i+i*\$	$E \rightarrow TE'$
TE'\$		

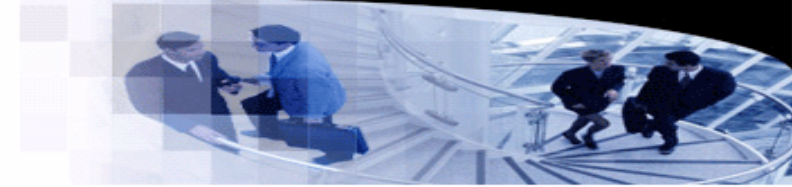
$select(E, i) = E \rightarrow TE'$.

LL(1) Parsing

- Table-driven parsing

- Example of LL(1) parsing:

	i	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow i$			$F \rightarrow (E)$		



Stack	Input	Action
E\$	i+i*i\$	$E \rightarrow TE'$
TE'\$	i+i*i\$	$T \rightarrow FT'$
FT'E'\$	i+i*i\$	$F \rightarrow i$
iT'E'\$	i+i*i\$	Match i
T'E'\$	+i*i\$	$T' \rightarrow \epsilon$
E'\$	+i*i\$	$E' \rightarrow +TE'$
+TE'\$	+i*i\$	Match +
TE'\$	i*i\$	$T \rightarrow FT'$
FT'E'\$	i*i\$	$F \rightarrow i$
iT'E'\$	i*i\$	Match i
T'E'\$	*i\$	$T' \rightarrow *FT'$
*FT'E'\$	*i\$	Match *
FT'E'\$	i\$	

- Example of LL(1) parsing:

[illegible][illegible]